

CM3065 Intelligent Signal Processing Coursework – Final Coursework 2025

Student Name: Devon Maya Xavier

UOL Student ID: 200618744

SIM Student ID: 10204322

Email: dmxavior001@mymail.sim.edu.sg

EXERCISE 1

Task 2 Table:

	Total number of cars	Cars per minute
Traffic_Laramie_1.mp4	6	2.02
Traffic_Laramie_2.mp4	5	2.84

Frame Differencing and Background Subtraction

Frame Differencing is a basic motion detection technique that involves subtracting consecutive frames of a video to identify moving objects, in this case, moving cars. This approach highlights differences between frames, making it effective to detect motion in relatively stable background environments. However, this technique is sensitive to lighting changes and camera vibrations, the prior being apparent when the green boxes surrounding the detected cars tend to flicker when the lighting changes on the cars.

In my code, I first convert the frames to grayscale to reduce computational complexity.

```
# Convert first frame to grayscale
prev_gray = cv2.cvtColor(prev_frame, cv2.COLOR_BGR2GRAY)
```

Then, I narrowed down the region of interest (ROI) to the bottom 55% of the frame.

```
# Define the starting position of the detection area (bottom 55% of the screen)
roi_start = int(prev_gray.shape[0] * 0.45)
```

The actual frame differencing is performed using the absolute difference function, and then filter out minor fluctuations by converting difference image into binary mask to emphasize regions with substantial motion.

```
# Frame Differencing
prev_roi = prev_gray[roi_start:, :]
frame_diff = cv2.absdiff(prev_roi, roi) # Compute difference between consecutive frames
_, frame_diff = cv2.threshold(frame_diff, 25, 255, cv2.THRESH_BINARY)
```

As for Background Subtraction, it takes on a different approach by creating and continuously update the MOG2 algorithm in the background.

```
# Initialize background subtractor
fgbg = cv2.createBackgroundSubtractorMOG2(history=500, varThreshold=50, detectShadows=True)
```

The method works here by learning a statistical model of the background over the last 500 frames. The `varThreshold` parameter determines the sensitivity of the detection – a higher threshold will mean that only larger deviations from the model are considered foreground. When we add the subtractor to ROI, the result is a mask where moving objects are highlighted against the background.

```
# Background Subtraction
fgmask = fgbg.apply(roi)
```

By combining these 2 techniques, we can quickly detect sudden changes between consecutive frames using Frame differencing and adapt to gradual changes in the environment using Background subtraction, which allows us to detect the moving cars.

```
# Combine both techniques
combined_mask = cv2.bitwise_or(fgmask, frame_diff)
```

Last but not least, Morphological operations are applied to further clean up the mask and then contours are extracted, which are later filtered by aspect ratio to distinguish vehicles from other objects (i.e., people) based on expected size and shape.

Analysis of the application

For Task 1, although the moving cars are detected, the detection boxes are not tagged to the cars when they are temporarily stationary, which is not useful as they will be recognised as a new car once they start moving again. That is something I need to further troubleshoot so that each car will be uniquely identified and not counted more than once if the application calls for car count within a frame rate.

For Task 2, on top of the Frame differencing and Background Subtractor techniques, a Centroid Tracker class is introduced to maintain persistent IDs for detected vehicles across the frames. An “Exit Zone” is created on the bottom left part of the frame to detect cars going towards the city centre (i.e., moving left). To avoid counting vehicles that might be stationary or moving in the wrong direction (for instance, a car moving from left to right that stops temporarily in the exit zone), the tracker stores a short history of x-coordinates for each tracked object in “Motion History”. When evaluating whether to count a vehicle, the code checks if its recorded x-positions have been consistently decreasing. This combined with the Frame differencing and Background subtractor techniques, allow the application to monitor how many cars driving towards the city centre.

EXERCISE 2

<https://hub.labs.coursera.org:443/connect/sharedhjvxsxpd?forceRefresh=false&isLabVersioning=true>

Analysis of Application

The application is a lossless audio compressor inspired by FLAC, using a second-order linear predictor to compute residuals, followed by Rice coding to encode the residuals.

The application seems to work better on simple signals (i.e., Sound1.wav voice audio), which shows it works for low-entropy, predictable audio like speech, and less effective on music audio with higher frame scale.

There is currently no stereo optimisation yet, and could likely be the cause of why compression is less effective on sound2.wav music file. With fixed k values of 2 and 4, there is no adaptivity to adjust to residual distribution. K=2 often performs poorly because it is too small for larger residuals.

Table and Results

	Original size (bytes)	Rice (K = 4 bits) (bytes)	Rice (K = 2 bits) (bytes)	% Compression (K = 4 bits)	% Compression (K = 2 bits)
Sound1.wav	1002088	565193	1063174	43.6	-6.10
Sound2.wav	1002088	56546703	225021309	-5509.55	-22222.57

The compression results for Sound1.wav and Sound2.wav using the same rice encoding show stark differences.

This is perhaps due to the fact that both audio files have different types of audio content and predictability. Sound1.wav is a speech audio file, which typically has long periods of low amplitude (silence or soft sounds) interspersed with bursts of higher amplitude (voiced segments). This results in a signal with relatively smooth transitions and lower variance over short time scales.

```
# Second-order predictor
def compute_residuals(samples):
    residuals = np.zeros_like(samples, dtype=np.int32)
    residuals[0] = samples[0]
    if len(samples) > 1:
        residuals[1] = samples[1] - samples[0]
    for i in range(2, len(samples)):
        predicted = 2 * samples[i-1] - samples[i-2]
        residuals[i] = samples[i] - predicted
    return residuals
```

The second-order predictor works well here because speech often allows a simple autoregressive pattern. Residuals are small, typically in the tens to hundreds, making them efficient to encode with Rice coding at k = 4.

Whereas for Sound2.wav, which is music at fast tempo of 168bpm, has complex waveforms with rapid changes due to beats, multiple instruments and harmonics. This creates a signal with high variance and less predictable patterns over short time scales.

The second-order predictor here struggles because music doesn't follow a simple linear trend. Residuals are found to be quite large in Sound2.wav, where transitions are abrupt. Large residuals produce long unary parts in Rice coding (i.e., thousands of zeros for q), inflating the bit stream dramatically.

	When k = 4	When k = 2
Sound1.wav	U = 100, q = 6, r = 4; Bits = $6 + 1 + 4 = 11$ (<16 bits original) = efficient	U = 100, q = 25, r = 0; Bits = $25 + 1 + 2 = 28$ (>16bits) = inefficient
Sound2.wav	U = 2000, q = 125, r=0; Bits = $125 + 1 + 4 = 130$ (>>16bits) = disastrous	U = 2000, q = 500, r = 0; Bits = $500 + 1 + 2 = 503$ (>>16bits) = even worse

Further Development Implemented

To see if I could further enhance the current model, I tested out by improving on the current model's Rice algorithm and also adopting another lossless data compression method – Huffman.

For the first, Stereo Decorrelation/Correlation is introduced, a fourth-order LPC predictor is applied to retrieve residuals instead of the previous second-order predictor. I also used Adaptive Rice coding which estimates the optimal k value per block based on the average logbased2 of the absolute residuals. Block processing is also added to enhance the model's efficiency. The results are shown below:

	Original Size	After compression size	Compression %
Sound1.wav	1002088	622264	37.90
Sound2.wav	1008044	1115029	-10.61

The results are significantly better especially for Sound2.wav, which shows to be rather promising.

As for the Huffman model, the residuals are retrieved through the compute_residuals function, with predictors 1,2 and 3. The choose_best_order function then calculates the total absolute residual error for each order and selects the one with the lowest error. This is then used for the entire file. The residuals are then encoded using Huffman coding. A unique counter is used in the heap to break ties when building the Huffman tree. The results are below:

	Original Size	Huffman (Order=2)	Huffman (Order=1)	Compression % (Order=2)	Compression % (Order=1)
Sound1.wav	1002088	466967	500923	53.4	50.01
Sound2.wav	1008044	1767009	1389802	-75.29	-37.87

For Sound1.wav, the compression model works well, achieving around 50–53% compression using both order-1 and order-2 predictors. In contrast, Sound2.wav shows negative compression ratios (–37.87% to –75.29%), meaning the compressed file is larger than the original. This suggests that the high-tempo, more complex music in Sound2.wav produces less predictable residuals, reducing the effectiveness of both the predictor and subsequent Huffman coding, and the overflow warning hints at potential issues with the predictor handling dynamic signal ranges.

EXERCISE 3

<https://hub.labs.coursera.org:443/connect/sharedhjvqvsxpd?forceRefresh=false&isLabVersioning=true>

How FFPROBE and FFMPEG are Installed and Configured

For this application, we used ffprobe and ffmpeg to analyse and convert video files according to the festival organisation's requirements.

The tools were installed using package managers (i.e., brew install ffmpeg) for Mac; ffprobe was used to extract metadata from the video files to check for compliance and ffmpeg was used to convert the non-compliant videos by modifying the container formats, codecs, resolutions, frame rate and bit rates.

Analysis of Application

The application uses function "get_video_metadata(video_path) by utilising ffprobe to extract video properties relevant to the required formats, then the metadata collected is displayed to show details of the video formats of each file.

We then use function "check_compliance_and_generate_report(films_metadata_df) to iterate through each video's metadata to compare it against the required specifications. If a video does not comply, the function records the problematic fields in a structured TXT report (format_compliance_report.txt) which is stored in the same directory.

Subsequently, the function "convert_non_compliant_films(films_metadata_df, folder_path) will proceed to automatically convert all non-compliant videos using ffmpeg. The script identifies and constructs an ffmpeg command to correct them.

- **c:v libx264**: Converts the video codec to H.264.
- **b:v 3M**: Adjusts the video bit rate to 3 Mb/s (within 2–5 Mb/s).
- **vf scale=640:360**: Resizes the video to 640x360 resolution.
- **r 25**: Ensures a frame rate of 25 FPS.
- **aspect 16:9**: Corrects the aspect ratio to 16:9.
- **c:a aac -b:a 256k -ac 2**: Converts audio codec to AAC, sets bit rate to ≤256 kb/s, and ensures stereo audio.

The converted files are then saved as new files with "_formatOK" appended to the filename, while the original files are preserved.

The application then does a final verification by comparing the original and converted files' metadata to ensure that conversion has been genuinely modified and not just its metadata information changed.

Description of Format terms

Video Format (container)	The digital "wrapper" that holds the video stream, audio stream, subtitles, and metadata (such as timestamps, bitrate, and codec information). Containers determine a file's compatibility with different media players and devices. Common containers are: MPEG-4 (MP4), Audio Video Interleave (AVI), Matroska Video (MKV), QuickTime File Format (MOV), Windows Media Video (WMV), WebM etc.
Video Codec	A video codec (short for coder-decoder) is the compression algorithm used to encode and decode digital video files. It reduces file size while preserving visual quality. Common codecs are: H.264 (AVC-Advanced Video Coding), H.265 (HEVC – High-Efficiency Video Coding), VP9, AV1 etc.
Audio Codec	An audio codec compresses and encodes sound in a digital format, reducing file size while maintaining quality. Common ones are: AAC (Advanced Audio Codec), MP3 (MPEG Audio Layer-3), Opus, FLAC (Free Lossless Audio Codec), Dolby Digital (AC3) etc.

Frame Rate	The frame rate (FPS - Frames Per Second) determines how many images (frames) are displayed per second in a video. A higher frame rate results in smoother motion. Common frame rates are: - 24 FPS – standard for movie and cinematic content - 25 FPS – Used in PAL (European) television broadcasting - 30 FPS – Common for NTSC (North American) TV and Youtube videos - 60 FPS – Used for high-quality gaming and sports broadcasts. - 20+ FPS – used in high-end gaming and VR applications.
Aspect Ratio	The proportional relationship between width and height of a video (e.g., 16:9 for widescreen format). Common aspect ratios are: - 4:3 (Standard Definition, Older TV Screens) - 16:9 (Widescreen, HD, and 4K Content) - 21:9 (Ultrawide, Cinematic Experience) - 1:1 (Square, Used for Social Media like Instagram) - 9:16 (Vertical Video, Optimized for Mobile Platforms like TikTok)
Resolution	Resolution refers to the number of pixels in a video, impacting its clarity and detail. Common resolutions: - 480p (SD - Standard Definition): 640x480 - 720p (HD - High Definition): 1280x720 - 1080p (Full HD): 1920x1080 - 4K (Ultra HD): 3840x2160 - 8K: 7680x4320
Video Bit Rate	The video bit rate determines how much data is used per second to encode video, measured in megabits per second (Mb/s). Common ranges for videos: - 1–2 Mb/s: Low-quality web video. - 2–5 Mb/s: Standard HD video (YouTube, streaming). - 10–20 Mb/s: High-quality Blu-ray video. - 50+ Mb/s: 4K HDR content.
Audio Bit Rate	The audio bit rate defines how much data is used to store sound, measured in kilobits per second (kb/s). Common ranges for audio: - 64 kb/s: Low-quality audio (VoIP, podcasts). - 128 kb/s: Standard MP3 quality. - 192 kb/s - 256 kb/s: High-quality streaming. - 320 kb/s: Highest MP3 quality - 1000+ kb/s: Lossless formats (FLAC, WAV).
Audio Channels	Defines the number of distinct audio outputs. Common configurations: - Mono (1 Channel) – Same sound in both speakers. - Stereo (2 Channels) – Separate left and right audio, creating depth. - 5.1 Surround (6 Channels) – Used in home theaters and cinemas. - 7.1 Surround (8 Channels) – More immersive sound, used in high-end audio systems.

EXERCISE 4

3 EMERGING COMPUTER VISION APPLICATIONS

1. AUTONOMOUS VEHICLES AND COMPUTER VISION IN TRANSPORTATION

Autonomous vehicles and Advanced Driver Assistance Systems (ADAS) rely heavily on computer vision for real-time object detection, lane tracking, pedestrian recognition, and obstacle avoidance. These systems integrate multiple sensors, including cameras, LiDAR, and radar, to interpret their environment and navigate safely.

Semantic segmentation techniques, such as those employed in Vision Transformers (ViTs), allow autonomous vehicles to analyse complex road scenes by labelling every pixel in an image. This capability supports ADAS features like automatic braking, adaptive cruise control, and lane-keeping assistance.

Beyond self-driving capabilities, intelligent traffic monitoring systems utilize AI-driven computer vision to optimize traffic flow, predict congestion patterns, and automate toll collection and law enforcement through license plate recognition.

2. MODERN SECURITY SYSTEMS

Computer vision has revolutionized security by enabling automated surveillance, threat detection, and anomaly recognition in both public and private spaces. AI-powered security systems integrate high-resolution cameras, deep learning models, and real-time processing to identify unauthorized access, suspicious behaviours, and potential threats.

Key applications include:

- **Intruder detection:** where vision-based models track movement patterns to prevent unauthorized access, and **facial recognition**, which ensures identity verification in restricted areas such as airports and corporate offices.
- **Weapon and fire detection algorithms:** enhance public safety by rapidly identifying potential threats in high-risk locations.
- **Smart city surveillance:** leverages computer vision to monitor traffic patterns, detect accidents, and analyse crowd behaviour in real time, improving urban security and emergency response.

Technologies like Edge AI enable on-device processing for faster decision-making, while deep learning models such as YOLO and Vision Transformers (ViTs) drive real-time object recognition with remarkable accuracy.

3. MEDICAL IMAGING AND HEALTHCARE

In the medical field, particularly in radiology, pathology, and disease diagnosis, AI-powered medical imaging systems now assist radiologists and pathologists in diagnosing diseases with unprecedented precision, reducing human error and expediting critical diagnoses.

Deep learning models, such as Convolutional Neural Networks (CNNs) and Vision Transformers (ViTs), are widely used for tumour detection, diabetic retinopathy screening, and histopathology analysis.

Computer vision-based image segmentation and classification are integral to analysing X-rays, MRIs, and CT scans, facilitating early detection of cancers, neurological disorders, and cardiovascular diseases. These innovations address key healthcare challenges, such as diagnostic delays, physician workload, and access to specialized care, making AI-powered diagnostics particularly valuable in remote and underdeveloped regions.

Beyond diagnostics, assistive technologies for visually impaired individuals are emerging, leveraging computer vision for obstacle detection, text reading, and facial recognition.

2 POPULAR STATE-OF-THE-ART COMPUTER VISION TECHNIQUES

1. VISION TRANSFORMERES (VITS)

Vision Transformers (ViTs) represent a major shift in computer vision, replacing Convolutional Neural Networks (CNNs) with a transformer-based architecture originally designed for natural language processing (NLP).

Unlike CNNs, which rely on localized receptive fields, ViTs process entire images globally by dividing them into fixed-size patches (e.g., 16×16 pixels), flattening these into sequences, and passing them through a transformer encoder with self-attention mechanisms. This allows ViTs to capture long-range dependencies across an image, making them highly effective for image classification, object detection, and segmentation.

The ViT architecture consists of several key components:

- **Patch embedding:** where the input image is divided and transformed into a sequence of vectors;
- **Positional encoding:** which retains spatial relationships;
- **Transformer encoder:** composed of multi-head self-attention layers and feed-forward networks; and
- **MLP head:** used for classification tasks.

The self-attention mechanism enables the model to selectively focus on different patches, allowing it to learn complex patterns and relationships within an image.

However, a key limitation of ViTs is their computational complexity, which scales quadratically with the number of patches ($O(n^2)$), making them less efficient for real-time applications like security surveillance and autonomous driving. To address this, recent research has focused on efficient attention mechanisms to reduce computational overhead, thereby expanding ViTs' applicability in domains like medical imaging, robotics, and smart city surveillance.

2. GENERATIVE ADVERSARIAL NETWORKS (GANS)

Generative Adversarial Networks (GANs) are a powerful deep learning technology used for creating and improving images. They consist of two competing neural networks:

- **Generator:** Creates new, synthetic images.
- **Discriminator:** Checks whether the images look real or fake.

These two networks compete in a minimax game, where the generator continuously improves, learning to create highly realistic images that can fool the discriminator.

GANs have several important uses in computer vision. For example, in super-resolution imaging, models like SRGANs can enhance blurry, low-quality images into high-resolution versions. This is especially useful in fields like medical imaging and satellite photography, where clarity is essential.

Another key application of GANs is data augmentation, where they generate synthetic training data to help AI models learn better. This is particularly valuable when there is limited real data, such as creating additional samples for fraud detection or diagnosing rare diseases.

POTENTIAL USES OF THE COMPUTER VISION & AUDIO FEATURE PROCESSING

BABY/TODDLER LANGUAGE INTERPRETATION

Computer vision and audio feature processing can play a transformative role in interpreting baby and toddler communication, particularly in early language development and sign recognition.

Infants and toddlers often use gestures, facial expressions, and vocalizations to express needs before they develop full speech. Computer vision can analyse hand movements, facial expressions, and body posture to recognize baby signs, which are widely used in early communication strategies to bridge the gap between understanding and speaking.

Deep learning models trained on annotated baby gesture datasets can identify common baby signs, such as signals for hunger, sleep, or discomfort, enabling real-time interpretation through smart cameras or mobile applications.

Simultaneously, audio feature processing can analyse cry patterns, tone variations, and early speech attempts, helping caregivers and healthcare professionals differentiate between different types of distress or communication attempts.

Advanced multi-modal AI models can combine visual and auditory cues to create context-aware systems that provide insights into an infant's emotional and physical state. Such technology has significant potential for parents, speech therapists, and paediatric specialists, aiding in early diagnosis of speech delays, autism spectrum disorders, or hearing impairments. Future advancements could see AI-powered baby monitors and assistive devices providing real-time feedback, helping parents and educators foster more effective early language development.

TURN SIGN LANGUAGE INTO MUSIC

Wearable-tech gloves can already translate sign language into speech, but using sign language to create real-time music is a new idea. This system would let deaf musicians and performers turn gestures into music using AI and computer vision. It would track hand shapes, arm movements, and facial expressions to interpret signs, then map them to musical elements—fast movements increase tempo, smooth motions create flowing notes, and sharp gestures make short, crisp sounds. Facial expressions would adjust emotion and intensity, making the music more expressive. The system would generate live MIDI music, allowing performers to “compose” through signing while also syncing with lights and visuals for immersive performances. Unlike other text-to-music or motion-based tools, this would create a new way for deaf artists to experience and create music.

REFERENCES

- Battineni, G., Chintalapudi, N., & Amenta, F. (2021). *Generative adversarial network: An overview of theory and applications*. *Cognitive Robotics*, 1, 19-26. <https://doi.org/10.1016/j.cogr.2020.12.001>
- V. K. Kukkala, J. Tunnell, S. Pasricha and T. Bradley, "Advanced Driver-Assistance Systems: A Path Toward Autonomous Vehicles," in *IEEE Consumer Electronics Magazine*, vol. 7, no. 5, pp. 18-25, Sept. 2018, doi: 10.1109/MCE.2018.2828440.
- Zhang, L., Lu, J., Zheng, S. *et al.* Vision Transformers: From Semantic Segmentation to Dense Prediction. *Int J Comput Vis* 132, 6142–6162 (2024). <https://doi.org/10.1007/s11263-024-02173-w>

APPENDIX

EXERCISE 1 – TASK 1: CODE

```
import cv2

import numpy as np

from scipy.spatial import distance

# Load video file
video_path = "Traffic_Laramie_1.mp4"
cap = cv2.VideoCapture(video_path)

# Initialize background subtractor
fgbg = cv2.createBackgroundSubtractorMOG2(history=500, varThreshold=50,
detectShadows=True)

# Define kernel
kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))

# Read the first frame for frame differencing
ret, prev_frame = cap.read()

if not ret:
    print("Error: Unable to read video")
```

```

cap.release()

exit()

# Convert first frame to grayscale
prev_gray = cv2.cvtColor(prev_frame, cv2.COLOR_BGR2GRAY)

# Define the starting position of the detection area (bottom 55% of the screen)
roi_start = int(prev_gray.shape[0] * 0.45)

# Dictionary to track cars {ID: (x, y, w, h, stationary_frames)}
tracked_cars = {}
next_car_id = 0
max_stationary_frames = 180 # Assuming 30 FPS, 6 seconds means 180 frames

while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break

    # Convert current frame to grayscale
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Select Only the Bottom 55%
    roi = gray[roi_start:, :]
    roi_frame = frame[roi_start:, :]

    # Frame Differencing
    prev_roi = prev_gray[roi_start:, :]

    frame_diff = cv2.absdiff(prev_roi, roi) # Compute difference between consecutive
frames

```

```

_, frame_diff = cv2.threshold(frame_diff, 25, 255, cv2.THRESH_BINARY)

# Background Subtraction
fgmask = fgbg.apply(roi)

# Combine both techniques
combined_mask = cv2.bitwise_or(fgmask, frame_diff)

# Remove noise
combined_mask = cv2.morphologyEx(combined_mask, cv2.MORPH_OPEN, kernel,
iterations=2)
combined_mask = cv2.morphologyEx(combined_mask, cv2.MORPH_DILATE, kernel,
iterations=2)

# Find contours
contours, _ = cv2.findContours(combined_mask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

detected_cars = []

# Process detected objects
for cnt in contours:
    area = cv2.contourArea(cnt)
    if area > 1200: # Ignore small rectangles (anything below 1200 pixels)
        x, y, w, h = cv2.boundingRect(cnt)
        y_full = y + roi_start # Convert y position to full-frame coordinates

# Exclude People
aspect_ratio = w / float(h)
if 1.5 < aspect_ratio < 4.5: # Typical aspect ratio for cars
    detected_cars.append((x, y_full, w, h))

```

```

# Match Detected Cars to Existing Tracked Cars

updated_tracked_cars = {}

used_ids = set()

for x, y, w, h in detected_cars:
    matched = False

    for car_id, (tx, ty, tw, th, stationary_frames) in tracked_cars.items():
        dist = distance.euclidean((x, y), (tx, ty))

        if dist < 50 and car_id not in used_ids: # If detected car is close to a
previously tracked car

            if (x, y) == (tx, ty): # If the car has stopped
                stationary_frames += 1
            else:
                stationary_frames = 0

            updated_tracked_cars[car_id] = (x, y, w, h, stationary_frames)
            used_ids.add(car_id)
            matched = True
            break

    if not matched:
        updated_tracked_cars[next_car_id] = (x, y, w, h, 0)
        used_ids.add(next_car_id)
        next_car_id += 1

# Remove cars that left the 55% region

tracked_cars = {
    car_id: (x, y, w, h, stationary_frames)
    for car_id, (x, y, w, h, stationary_frames) in updated_tracked_cars.items()
}

```

```

        if y + h > roi_start # Remove cars that left the detection area
    }

    # Draw 55% Detection Boundary Line
    for i in range(0, frame.shape[1], 20): # Draw a dashed red line
        cv2.line(frame, (i, roi_start), (i + 10, roi_start), (0, 0, 255), 2)

    # Add "Main Street" label above the line
    cv2.putText(frame, "Main Street", (10, roi_start - 10), cv2.FONT_HERSHEY_SIMPLEX,
0.7, (0, 0, 255), 2)

    # Draw Bounding Boxes for Tracked Cars
    for car_id, (x, y, w, h, stationary_frames) in tracked_cars.items():
        cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2) # Keep the box
fixed on detected cars

    # Display the processed frame
    cv2.imshow("Car Detection", frame)

    # Update previous frame for next iteration
    prev_gray = gray.copy()

    # Break loop on 'q' key press
    if cv2.waitKey(30) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()

```

EXERCISE 1 – TASK 2: CODE

```
import cv2
```

```

import numpy as np

import math

# Initialize background subtractor (MOG2 algorithm)

bg_subtractor = cv2.createBackgroundSubtractorMOG2(history=500, varThreshold=50,
detectShadows=False)

# Define exit zone parameters

EXIT_ZONE_PERCENT_HEIGHT = 0.48 # bottom 48%

EXIT_ZONE_PERCENT_WIDTH = 0.04 # left 4%

MOTION_THRESHOLD = 3 # Vehicle must move left for at least 3 consecutive frames before
counting

class CentroidTracker:

    def __init__(self, dist_threshold=25):

        self.objects = {}

        self.next_id = 0

        self.dist_threshold = dist_threshold

        self.initial_x = {}

        self.counted_ids = set()

        self.motion_history = {}

    def update(self, detections):

        objects_with_id = []

        new_centroids = []

        for (x, y, w, h) in detections:

            cx = x + w//2

            cy = y + h//2

            new_centroids.append((cx, cy, x, y, w, h))

        updated_ids = {}

```

```

for (cx, cy, x, y, w, h) in new_centroids:

    match_id = None

    min_dist = self.dist_threshold

    for obj_id, (prev_cx, prev_cy) in self.objects.items():

        dist = math.hypot(cx - prev_cx, cy - prev_cy)

        if dist < min_dist:

            min_dist = dist

            match_id = obj_id

    if match_id is not None:

        # Update existing object

        self.objects[match_id] = (cx, cy)

        updated_ids[match_id] = True

        objects_with_id.append([x, y, w, h, match_id])

        # Store recent x-positions to check consistent leftward motion

        if match_id not in self.motion_history:

            self.motion_history[match_id] = []

        self.motion_history[match_id].append(cx)

        # Keep only the last motion threshold positions

        if len(self.motion_history[match_id]) > MOTION_THRESHOLD:

            self.motion_history[match_id].pop(0)

    else:

        # Register new object

        obj_id = self.next_id

        self.objects[obj_id] = (cx, cy)

        self.initial_x[obj_id] = cx

```



```

        objects_with_id.append([x, y, w, h, obj_id])

        self.next_id += 1

    # Initialize motion history

    self.motion_history[obj_id] = [cx]

    current_ids = set([obj[-1] for obj in objects_with_id])

    disappeared_ids = [obj_id for obj_id in list(self.objects.keys()) if obj_id not
in current_ids]

    for obj_id in disappeared_ids:

        self.objects.pop(obj_id, None)

        self.initial_x.pop(obj_id, None)

        self.motion_history.pop(obj_id, None) # Remove motion history when object
disappears

    return objects_with_id

def count_vehicles(video_path):

    cap = cv2.VideoCapture(video_path)

    if not cap.isOpened():

        print(f"Unable to open video: {video_path}")

        return 0

    frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))

    frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

    fps = cap.get(cv2.CAP_PROP_FPS) # Get video frame rate

    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT)) # Total frames in video

    total_seconds = total_frames / fps # Total duration of video in seconds

    total_minutes = total_seconds / 60 # Total duration in minutes

```

```

exit_zone_x = int(EXIT_ZONE_PERCENT_WIDTH * frame_width)
exit_zone_y = int((1.0 - EXIT_ZONE_PERCENT_HEIGHT) * frame_height)

tracker = CentroidTracker(dist_threshold=30)
total_count = 0

while True:
    ret, frame = cap.read()
    if not ret:
        break

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    fg_mask = bg_subtractor.apply(gray)

    kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (5,5))
    mask_clean = cv2.morphologyEx(fg_mask, cv2.MORPH_CLOSE, kernel)
    mask_clean = cv2.morphologyEx(mask_clean, cv2.MORPH_OPEN, kernel)
    mask_clean = cv2.dilate(mask_clean, kernel, iterations=2)

    _, mask_clean = cv2.threshold(mask_clean, 127, 255, cv2.THRESH_BINARY)

    contours, _ = cv2.findContours(mask_clean, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

    detections = []
    for cnt in contours:
        area = cv2.contourArea(cnt)
        if area < 500:
            continue
        x, y, w, h = cv2.boundingRect(cnt)

```

```

        detections.append((x, y, w, h))

objects = tracker.update(detections)

for (x, y, w, h, obj_id) in objects:
    cx = x + w//2
    cy = y + h//2
    if obj_id not in tracker.counted_ids:
        # Check if object is in exit zone and moving left for motion threshold
frames
        if (cx < exit_zone_x and cy > exit_zone_y and
tracker.initial_x.get(obj_id, frame_width) > exit_zone_x):
            x_positions = tracker.motion_history.get(obj_id, [])
            if len(x_positions) >= MOTION_THRESHOLD and all(x_positions[i] >
x_positions[i + 1] for i in range(len(x_positions) - 1)):
                total_count += 1
                tracker.counted_ids.add(obj_id)

    cv2.rectangle(frame, (0, exit_zone_y), (exit_zone_x, frame_height), (0, 0, 255),
2)

    cv2.putText(frame, "Exit Zone", (10, exit_zone_y - 10),
cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 0, 255), 2)

    cv2.putText(frame, f"Count: {total_count}", (10, 30), cv2.FONT_HERSHEY_SIMPLEX,
1.0, (255, 255, 255), 2)

    cv2.imshow("Vehicle Counter", frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()

cv2.destroyAllWindows()

```

```

avg_cars_per_min = total_count / total_minutes if total_minutes > 0 else 0

print(f"Total vehicles counted: {total_count}")

print(f"Total video duration: {total_minutes:.2f} minutes")

print(f"Average vehicles per minute: {avg_cars_per_min:.2f}")

return total_count

```

```

# Run the function on video files

count1 = count_vehicles('Traffic_Laramie_1.mp4')

print("Vehicles exited left (Traffic_Laramie_1):", count1)

count2 = count_vehicles('Traffic_Laramie_2.mp4')

print("Vehicles exited left (Traffic_Laramie_2):", count2)

```

EXERCISE 2: CODE – SOUND1.WAV

```

import wave

import struct

import os

import numpy as np

from bitarray import bitarray

# Read WAV file

def read_wav_file(filename):

    with wave.open(filename, 'rb') as wav_file:

        num_channels = wav_file.getnchannels()

        sample_width = wav_file.getsampwidth()

        frame_rate = wav_file.getframerate()

        num_frames = wav_file.getnframes()

        frames = wav_file.readframes(num_frames)

        samples = np.frombuffer(frames, dtype=np.int16)

```

```
    return samples, num_channels, frame_rate, sample_width
```

```
# Second-order predictor
```

```
def compute_residuals(samples):
```

```
    residuals = np.zeros_like(samples, dtype=np.int32)
```

```
    residuals[0] = samples[0]
```

```
    if len(samples) > 1:
```

```
        residuals[1] = samples[1] - samples[0]
```

```
    for i in range(2, len(samples)):
```

```
        predicted = 2 * samples[i-1] - samples[i-2]
```

```
        residuals[i] = samples[i] - predicted
```

```
    return residuals
```

```
# Simplified mapping: absolute value + sign bit
```

```
def encode_residuals(residuals):
```

```
    abs_residuals = np.abs(residuals).astype(np.uint32)
```

```
    signs = (residuals < 0).astype(np.uint8) # 1 for negative, 0 for positive
```

```
    return abs_residuals, signs
```

```
# Rice encoding
```

```
def rice_encode(u, k):
```

```
    q = u >> k
```

```
    r = u & ((1 << k) - 1)
```

```
    unary = bytearray('0' * q + '1')
```

```
    binary = bytearray(format(r, f'0{k}b'))
```

```
    return unary + binary
```

```
# Encode to file with sign bits
```

```
def encode_to_file(samples, k, output_filename):
```

```
    residuals = compute_residuals(samples)
```

```

abs_residuals, signs = encode_residuals(residuals)

num_samples = len(residuals)

bit_stream = bytearray()

for i in range(num_samples):
    bit_stream.append(signs[i]) # Add sign bit
    bit_stream.extend(rice_encode(abs_residuals[i], k)) # Encode magnitude

if len(bit_stream) % 8:
    bit_stream.extend('0' * (8 - len(bit_stream) % 8))

header = struct.pack('>I B', num_samples, k)
with open(output_filename, 'wb') as f:
    f.write(header)
    f.write(bit_stream.tobytes())

return len(header) + len(bit_stream.tobytes())

```

Decode from file

```

def decode_from_file(filename):
    with open(filename, 'rb') as f:
        header = f.read(5)
        num_samples, k = struct.unpack('>I B', header)
        bit_data = f.read()
        bits = bytearray()
        bits.frombytes(bit_data)

    abs_residuals = []
    signs = []
    pos = 0

```

```

bits_str = bits.to01()
while pos < len(bits_str) and len(abs_residuals) < num_samples:

    # Read sign bit

    sign = int(bits_str[pos])

    signs.append(sign)

    pos += 1

    # Read Rice-coded magnitude

    q = 0

    while pos < len(bits_str) and bits_str[pos] == '0':

        q += 1

        pos += 1

    if pos >= len(bits_str):

        break

    pos += 1 # Skip '1'

    if pos + k > len(bits_str):

        break

    r = int(bits_str[pos:pos + k], 2)

    pos += k

    u = (q << k) + r

    abs_residuals.append(u)

# Reconstruct residuals with signs

residuals = np.where(np.array(signs) == 0, np.array(abs_residuals), -
np.array(abs_residuals)).astype(np.int32)

# Reconstruct samples

samples = np.zeros_like(residuals, dtype=np.int16)

samples[0] = residuals[0]

if len(residuals) > 1:

```

```

        samples[1] = residuals[1] + samples[0]
    for i in range(2, len(residuals)):
        predicted = 2 * samples[i-1] - samples[i-2]
        samples[i] = predicted + residuals[i]

    return samples[:num_samples]

# Write WAV file
def write_wav_file(samples, num_channels, frame_rate, sample_width, output_filename):
    with wave.open(output_filename, 'wb') as wav_file:
        wav_file.setnchannels(num_channels)
        wav_file.setsampwidth(sample_width)
        wav_file.setframerate(frame_rate)
        wav_file.writeframes(samples.tobytes())

# Process audio file
def process_audio_file(input_filename, k_values=[2, 4]):
    samples, num_channels, frame_rate, sample_width = read_wav_file(input_filename)
    original_size = os.path.getsize(input_filename)
    results = {'Original Size': original_size}

    for k in k_values:
        encoded_filename = input_filename.replace('.wav', f'_Enc{k}.ex2')
        compressed_size = encode_to_file(samples, k, encoded_filename)
        results[f'Rice (K={k} bits)'] = compressed_size

        decoded_samples = decode_from_file(encoded_filename)
        decoded_filename = input_filename.replace('.wav', f'_Enc_Dec{k}.wav')
        write_wav_file(decoded_samples, num_channels, frame_rate, sample_width,
            decoded_filename)

```



```

        if np.array_equal(decoded_samples, samples):
            print(f"Lossless compression verified for {input_filename} with k={k}")
        else:
            print(f"Warning: Lossless compression failed for {input_filename} with
k={k}")

            print(f"First 10 original samples: {samples[:10]}")
            print(f"First 10 decoded samples: {decoded_samples[:10]}")

    for k in k_values:
        compressed_size = results[f'Rice (K={k} bits)']
        percent_compression = ((original_size - compressed_size) / original_size) * 100
        results[f'% Compression (K={k} bits)'] = percent_compression

    return results

# Example usage
input_files = ['Sound1.wav']
k_values = [2, 4]
table_data = {}

for input_file in input_files:
    if os.path.exists(input_file):
        results = process_audio_file(input_file, k_values)
        table_data[input_file] = results
    else:
        print(f"File {input_file} not found.")

# Print table
print("\nCompression Analysis Table:")

```

```

print("| Audio File | Original Size | Rice (K=4 bits) | Rice (K=2 bits) | % Compression  
(K=4 bits) | % Compression (K=2 bits) |")

print("|-----|-----|-----|-----|-----  
-----|-----|")

for file, data in table_data.items():

    print(f"| {file:<11} | {data['Original Size']:>13} | {data['Rice (K=4 bits)']:>15} |  
{data['Rice (K=2 bits)']:>15} | {data['% Compression (K=4 bits)']:>23.2f} | {data['%  
Compression (K=2 bits)']:>23.2f} |")

```

EXERCISE 2: CODE – SOUNDWAV.2

```

import wave

import struct

import os

import numpy as np

from bitarray import bitarray

# Read WAV file

def read_wav_file(filename):

    with wave.open(filename, 'rb') as wav_file:

        num_channels = wav_file.getnchannels()

        sample_width = wav_file.getsampwidth()

        frame_rate = wav_file.getframerate()

        num_frames = wav_file.getnframes()

        frames = wav_file.readframes(num_frames)

        samples = np.frombuffer(frames, dtype=np.int16)

        return samples, num_channels, frame_rate, sample_width

# Second-order predictor

def compute_residuals(samples):

    residuals = np.zeros_like(samples, dtype=np.int32)

    residuals[0] = samples[0]

```

```

if len(samples) > 1:
    residuals[1] = samples[1] - samples[0]
for i in range(2, len(samples)):
    predicted = 2 * samples[i-1] - samples[i-2]
    residuals[i] = samples[i] - predicted
return residuals

# Simplified mapping: absolute value + sign bit
def encode_residuals(residuals):
    abs_residuals = np.abs(residuals).astype(np.uint32)
    signs = (residuals < 0).astype(np.uint8) # 1 for negative, 0 for positive
    return abs_residuals, signs

# Rice encoding
def rice_encode(u, k):
    q = u >> k
    r = u & ((1 << k) - 1)
    unary = bytearray('0' * q + '1')
    binary = bytearray(format(r, f'0{k}b'))
    return unary + binary

# Encode to file with sign bits
def encode_to_file(samples, k, output_filename):
    residuals = compute_residuals(samples)
    abs_residuals, signs = encode_residuals(residuals)
    num_samples = len(residuals)

    bit_stream = bytearray()
    for i in range(num_samples):
        bit_stream.append(signs[i]) # Add sign bit

```

```

        bit_stream.extend(rice_encode(abs_residuals[i], k)) # Encode magnitude

if len(bit_stream) % 8:
    bit_stream.extend('0' * (8 - len(bit_stream) % 8))

header = struct.pack('>I B', num_samples, k)
with open(output_filename, 'wb') as f:
    f.write(header)
    f.write(bit_stream.tobytes())

return len(header) + len(bit_stream.tobytes())

# Decode from file
def decode_from_file(filename):
    with open(filename, 'rb') as f:
        header = f.read(5)
        num_samples, k = struct.unpack('>I B', header)
        bit_data = f.read()
        bits = bitarray()
        bits.frombytes(bit_data)

    abs_residuals = []
    signs = []
    pos = 0
    bits_str = bits.to01()
    while pos < len(bits_str) and len(abs_residuals) < num_samples:
        # Read sign bit
        sign = int(bits_str[pos])
        signs.append(sign)
        pos += 1

```

```

# Read Rice-coded magnitude

q = 0

while pos < len(bits_str) and bits_str[pos] == '0':

    q += 1

    pos += 1

if pos >= len(bits_str):

    break

pos += 1 # Skip '1'

if pos + k > len(bits_str):

    break

r = int(bits_str[pos:pos + k], 2)

pos += k

u = (q << k) + r

abs_residuals.append(u)


# Reconstruct residuals with signs

residuals = np.where(np.array(signs) == 0, np.array(abs_residuals), -
np.array(abs_residuals)).astype(np.int32)


# Reconstruct samples

samples = np.zeros_like(residuals, dtype=np.int16)

samples[0] = residuals[0]

if len(residuals) > 1:

    samples[1] = residuals[1] + samples[0]

for i in range(2, len(residuals)):

    predicted = 2 * samples[i-1] - samples[i-2]

    samples[i] = predicted + residuals[i]


return samples[:num_samples]

```

```

# Write WAV file

def write_wav_file(samples, num_channels, frame_rate, sample_width, output_filename):
    with wave.open(output_filename, 'wb') as wav_file:
        wav_file.setnchannels(num_channels)
        wav_file.setsampwidth(sample_width)
        wav_file.setframerate(frame_rate)
        wav_file.writeframes(samples.tobytes())

# Process audio file

def process_audio_file(input_filename, k_values=[2, 4]):
    samples, num_channels, frame_rate, sample_width = read_wav_file(input_filename)
    original_size = os.path.getsize(input_filename)
    results = {'Original Size': original_size}

    for k in k_values:
        encoded_filename = input_filename.replace('.wav', f'_Enc{k}.ex2')
        compressed_size = encode_to_file(samples, k, encoded_filename)
        results[f'Rice (K={k} bits)'] = compressed_size

        decoded_samples = decode_from_file(encoded_filename)
        decoded_filename = input_filename.replace('.wav', f'_Enc_Dec{k}.wav')
        write_wav_file(decoded_samples, num_channels, frame_rate, sample_width,
            decoded_filename)

        if np.array_equal(decoded_samples, samples):
            print(f"Lossless compression verified for {input_filename} with k={k}")
        else:
            print(f"Warning: Lossless compression failed for {input_filename} with
k={k}")

            print(f"First 10 original samples: {samples[:10]}")

```

```

        print(f"First 10 decoded samples: {decoded_samples[:10]}")

    for k in k_values:

        compressed_size = results[f'Rice (K={k} bits)']

        percent_compression = ((original_size - compressed_size) / original_size) * 100

        results[f'% Compression (K={k} bits)'] = percent_compression

    return results

# Example usage

input_files = ['Sound2.wav']

k_values = [2, 4]

table_data = {}

for input_file in input_files:

    if os.path.exists(input_file):

        results = process_audio_file(input_file, k_values)

        table_data[input_file] = results

    else:

        print(f"File {input_file} not found.")

# Print table

print("\nCompression Analysis Table:")

print("| Audio File | Original Size | Rice (K=4 bits) | Rice (K=2 bits) | % Compression  
(K=4 bits) | % Compression (K=2 bits) |")

print("|-----|-----|-----|-----|-----|  
-----|-----|-----|-----|-----|")

for file, data in table_data.items():

    print(f"| {file:<11} | {data['Original Size']:>13} | {data['Rice (K=4 bits)']:>15} |  
{data['Rice (K=2 bits)']:>15} | {data['% Compression (K=4 bits)']:>23.2f} | {data['%  
Compression (K=2 bits)']:>23.2f} |")

```

EXERCISE 2: CODE – FURTHER IMPROVEMENT

```
import wave

import struct

import os

import numpy as np

from bitarray import bitarray


# Read WAV file

def read_wav_file(filename):

    with wave.open(filename, 'rb') as wav_file:

        num_channels = wav_file.getnchannels()

        sample_width = wav_file.getsampwidth()

        frame_rate = wav_file.getframerate()

        num_frames = wav_file.getnframes()

        frames = wav_file.readframes(num_frames)

        samples = np.frombuffer(frames, dtype=np.int16)

        return samples, num_channels, frame_rate, sample_width


# Stereo decorrelation

def stereo_decorrelate(samples, num_channels):

    if num_channels == 2:

        left = samples[0::2]

        right = samples[1::2]

        mid = (left + right) // 2

        side = left - right

        return np.vstack((mid, side)).T.flatten()

    return samples


# Reverse stereo decorrelation
```



```

def stereo_correlate(samples, num_channels):
    if num_channels == 2:
        mid = samples[0::2]
        side = samples[1::2]
        left = mid + side
        right = mid - side
        return np.vstack((left, right)).T.flatten()
    return samples

# LPC predictor (order 4, vectorized)
def compute_lpc_residuals(samples, order=4):
    residuals = np.zeros_like(samples, dtype=np.int32)
    residuals[:order] = samples[:order]
    coeffs = np.array([0.95, -0.5, 0.3, -0.1]) # Optimized for general audio
    if len(samples) > order:
        past_samples = np.lib.stride_tricks.sliding_window_view(samples,
order)[::(len(samples)-order)]
        predicted = np.dot(past_samples, coeffs[::-1]).astype(np.int32)
        residuals[order:] = samples[order:] - predicted
    return residuals

# Reconstruct samples with LPC
def reconstruct_samples(residuals, order=4):
    samples = np.zeros_like(residuals, dtype=np.int16)
    samples[:order] = residuals[:order].astype(np.int16)
    coeffs = np.array([0.95, -0.5, 0.3, -0.1])
    if len(residuals) > order:
        past_samples = np.lib.stride_tricks.sliding_window_view(samples,
order)[::(len(samples)-order)]
        predicted = np.dot(past_samples, coeffs[::-1]).astype(np.int32)
        samples[order:] = (predicted + residuals[order:]).astype(np.int16)

```

```
    return samples
```

```
# Rice coding with adaptive k
```

```
def select_rice_k(residuals):
```

```
    abs_residuals = np.abs(residuals)
```

```
    mean_log2 = np.mean(np.log2(abs_residuals + 1)) # Estimate optimal k
```

```
    k = max(0, min(15, int(mean_log2 - 1))) # Range 0-15
```

```
    return k
```

```
def rice_encode(residuals, k):
```

```
    bit_stream = bytearray()
```

```
    for r in residuals:
```

```
        sign = 1 if r < 0 else 0
```

```
        u = abs(r)
```

```
        q = u >> k
```

```
        rem = u & ((1 << k) - 1)
```

```
        bit_stream.append(sign)
```

```
        bit_stream.extend('0' * q + '1')
```

```
        # For k==0, this adds an empty string (which is fine)
```

```
        bit_stream.extend(format(rem, f'0{k}b'))
```

```
    return bit_stream
```

```
def rice_decode(bit_stream, k, num_samples):
```

```
    residuals = []
```

```
    pos = 0
```

```
    while pos < len(bit_stream) and len(residuals) < num_samples:
```

```
        # Read sign bit
```

```
        sign = bit_stream[pos]
```

```
        pos += 1
```

```

q = 0

# Count the number of 0's until a 1 is reached (unary part)
while pos < len(bit_stream) and bit_stream[pos] == 0:

    q += 1

    pos += 1

if pos >= len(bit_stream):

    break

pos += 1 # Skip the '1'

# If k==0, no bits to read for remainder; set r_val = 0
if k == 0:

    r_val = 0

else:

    if pos + k > len(bit_stream):

        break

    r_val = int(bit_stream[pos:pos + k].to01(), 2)

    pos += k

value = (-1 if sign else 1) * ((q << k) + r_val)

residuals.append(value)

return np.array(residuals, dtype=np.int32)

```

Encode to file with block processing

```

def encode_to_file(samples, num_channels, output_filename, block_size=4096):

    preprocessed_samples = stereo_decorrelate(samples, num_channels)

    num_samples = len(preprocessed_samples)

    # Using LPC predictor of order 4

    residuals = compute_lpc_residuals(preprocessed_samples)

    bit_stream = bytearray()

    block_headers = []

    for start in range(0, num_samples, block_size):

```

```

        end = min(start + block_size, num_samples)

        block_residuals = residuals[start:end]

        k = select_rice_k(block_residuals)

        block_bit_stream = rice_encode(block_residuals, k)

        block_headers.append((k, len(block_bit_stream)))

        bit_stream.extend(block_bit_stream)

    if len(bit_stream) % 8:

        bit_stream.extend('0' * (8 - len(bit_stream) % 8))

    # Write header: 4 bytes num_samples, 1 byte num_channels, 4 bytes num_blocks
    header = struct.pack('>I B I', num_samples, num_channels, len(block_headers))
    total_size = len(header)

    with open(output_filename, 'wb') as f:

        f.write(header)

        for k, bits_len in block_headers:

            f.write(struct.pack('>B I', k, bits_len)) # k as 1 byte, bits_len as 4
bytes
            total_size += 5

        encoded_bytes = bit_stream.tobytes()

        f.write(encoded_bytes)

        total_size += len(encoded_bytes)

    return total_size

# Decode from file
def decode_from_file(filename):

    with open(filename, 'rb') as f:

        header = f.read(9)

        if len(header) < 9:

```

```

        raise ValueError(f"File {filename} too short: only {len(header)} bytes")
num_samples, num_channels, num_blocks = struct.unpack('>I B I', header)

block_headers = []
for _ in range(num_blocks):
    header_data = f.read(5)
    if len(header_data) < 5:
        raise ValueError("Incomplete block header")
    k, bits_len = struct.unpack('>B I', header_data)
    block_headers.append((k, bits_len))

bit_data = f.read()
bits = bytearray()
bits.frombytes(bit_data)

residuals = np.zeros(num_samples, dtype=np.int32)
bit_pos = 0
for i, block_start in enumerate(range(0, num_samples, 4096)):
    block_end = min(block_start + 4096, num_samples)
    k, block_bits_len = block_headers[i]
    block_bits = bits[bit_pos:bit_pos + block_bits_len]
    block_residuals = rice_decode(block_bits, k, block_end - block_start)
    residuals[block_start:block_end] = block_residuals
    bit_pos += block_bits_len

reconstructed = reconstruct_samples(residuals)
# Reverse the stereo decorrelation
return stereo_correlate(reconstructed, num_channels)

# Write WAV file

```

```

def write_wav_file(samples, num_channels, frame_rate, sample_width, output_filename):
    with wave.open(output_filename, 'wb') as wav_file:
        wav_file.setnchannels(num_channels)
        wav_file.setsampwidth(sample_width)
        wav_file.setframerate(frame_rate)
        wav_file.writeframes(samples.tobytes())

# Process audio file
def process_audio_file(input_filename):
    samples, num_channels, frame_rate, sample_width = read_wav_file(input_filename)
    original_size = os.path.getsize(input_filename)
    results = {'Original Size': original_size}

    encoded_filename = input_filename.replace('.wav', '_Enc.ex2')
    compressed_size = encode_to_file(samples, num_channels, encoded_filename)
    results['FLAC-like'] = compressed_size

    decoded_samples = decode_from_file(encoded_filename)
    decoded_filename = input_filename.replace('.wav', '_Enc_Dec.wav')
    write_wav_file(decoded_samples, num_channels, frame_rate, sample_width,
        decoded_filename)

    if np.array_equal(decoded_samples, samples):
        print(f"Lossless compression verified for {input_filename}")
    else:
        print(f"Warning: Lossless compression failed for {input_filename}")
        print(f"First 10 original samples: {samples[:10]}")
        print(f"First 10 decoded samples: {decoded_samples[:10]}")
        diff = np.where(samples != decoded_samples)[0]
        if len(diff) > 0:

```

```

        print(f"Differences at indices: {diff[:10]}")

        print(f"Sample values: {samples[diff[:10]]} vs
{decoded_samples[diff[:10]]}")

    percent_compression = ((original_size - compressed_size) / original_size) * 100
    results['% Compression'] = percent_compression

    return results

# Example usage
input_files = ['Sound1.wav', 'Sound2.wav']
table_data = {}

for input_file in input_files:
    if os.path.exists(input_file):
        results = process_audio_file(input_file)
        table_data[input_file] = results
    else:
        print(f"File {input_file} not found.")

# Print table
print("\nCompression Analysis Table:")
print("| Audio File | Original Size | FLAC-like (bytes) | % Compression |")
print("|-----|-----|-----|-----|")
for file, data in table_data.items():
    print(f"| {file:<11} | {data['Original Size']:>13} | {data['FLAC-like']:>17} |
{data['% Compression']:>13.2f} |")

```

EXERCISE 2: CODE – FURTHER IMPROVEMENT WITH HUFFMAN

```
import wave
```

```

import struct

import os

import numpy as np

from bitarray import bitarray

import heapq


# WAV File

def read_wav_file(filename):
    with wave.open(filename, 'rb') as wav_file:
        num_channels = wav_file.getnchannels()
        sample_width = wav_file.getsampwidth()
        frame_rate = wav_file.getframerate()
        num_frames = wav_file.getnframes()
        frames = wav_file.readframes(num_frames)
        samples = np.frombuffer(frames, dtype=np.int16)
        return samples, num_channels, frame_rate, sample_width

def write_wav_file(samples, num_channels, frame_rate, sample_width, output_filename):
    with wave.open(output_filename, 'wb') as wav_file:
        wav_file.setnchannels(num_channels)
        wav_file.setsampwidth(sample_width)
        wav_file.setframerate(frame_rate)
        wav_file.writeframes(samples.tobytes())


# Predictive Coding (Lossless)

def compute_residuals(samples, order):
    residuals = np.zeros_like(samples, dtype=np.int32)
    residuals[0] = samples[0]
    if len(samples) > 1:
        residuals[1] = samples[1] - samples[0]

```



```

for i in range(2, len(samples)):
    if order == 1:
        predicted = samples[i-1]
    elif order == 2:
        predicted = 2 * samples[i-1] - samples[i-2]
    else:
        # For orders > 2, fallback to order 1
        predicted = samples[i-1]
    residuals[i] = samples[i] - predicted
return residuals

```

Mapping residuals to nonnegative integers (and back)

```

def map_residual(r):
    return int(2 * r) if r >= 0 else int(-2 * r - 1)

```

```

def inverse_map_residual(u):
    return u // 2 if u % 2 == 0 else -(u + 1) // 2

```

Huffman Coding Helper Functions

```

def build_frequency_table(data):
    freq = {}
    for d in data:
        freq[d] = freq.get(d, 0) + 1
    return freq

```

```

def build_huffman_tree(freq):
    heap = []
    count = 0
    for symbol, frequency in freq.items():
        heapq.heappush(heap, (frequency, count, symbol))

```

```

        count += 1
while len(heap) > 1:
    freq1, count1, left = heapq.heappop(heap)
    freq2, count2, right = heapq.heappop(heap)
    new_node = (left, right)
    heapq.heappush(heap, (freq1 + freq2, count, new_node))
    count += 1
return heap[0][2]

```

```

def build_huffman_codes(tree, prefix=""):
    codes = {}
    if isinstance(tree, int):
        codes[tree] = prefix if prefix != "" else "0"
    else:
        left, right = tree
        codes.update(build_huffman_codes(left, prefix + "0"))
        codes.update(build_huffman_codes(right, prefix + "1"))
    return codes

```

```

def huffman_encode(data, codes):
    encoded = bytearray()
    for d in data:
        encoded.extend(codes[d])
    return encoded

```

```

def huffman_decode(encoded, tree, num_symbols):
    decoded = []
    node = tree
    for bit in encoded:
        node = node[0] if bit == 0 else node[1]

```

```

        if isinstance(node, int):

            decoded.append(node)

            node = tree

            if len(decoded) == num_symbols:

                break

    return decoded

```

File Compression with Huffman Coding

```
def encode_to_file_huffman(samples, order, output_filename):
```

```
    # Compute prediction residuals
```

```
    residuals = compute_residuals(samples, order)
```

```
    # Map residuals to nonnegative integers
```

```
    mapped = [map_residual(r) for r in residuals]
```

```
    num_samples = len(mapped)
```

```
    # Build frequency table and Huffman codes
```

```
    freq_table = build_frequency_table(mapped)
```

```
    tree = build_huffman_tree(freq_table)
```

```
    codes = build_huffman_codes(tree)
```

```
    # Encode the mapped residuals using Huffman codes
```

```
    encoded = huffman_encode(mapped, codes)
```

```
    # Ensure bit stream is byte-aligned
```

```
    if len(encoded) % 8:
```

```
        encoded.extend('0' * (8 - len(encoded) % 8))
```

```
    encoded_bytes = encoded.tobytes()
```

```
    header = struct.pack('>I B', num_samples, order)
```

```
    header += struct.pack('>I', len(freq_table))
```

```
    # For each entry: 4 bytes symbol, 4 bytes frequency
```

```

for symbol, frequency in freq_table.items():
    header += struct.pack('>I I', symbol, frequency)

with open(output_filename, 'wb') as f:
    f.write(header)
    f.write(encoded_bytes)

return len(header) + len(encoded_bytes)

def decode_from_file_huffman(filename):
    with open(filename, 'rb') as f:
        # Read basic header: number of samples and predictor order
        header = f.read(5)
        num_samples, order = struct.unpack('>I B', header)

        # Read frequency table size
        table_size_bytes = f.read(4)
        num_entries = struct.unpack('>I', table_size_bytes)[0]
        freq_table = {}

        for _ in range(num_entries):
            entry = f.read(8)
            symbol, frequency = struct.unpack('>I I', entry)
            freq_table[symbol] = frequency

        # Rebuild Huffman tree
        tree = build_huffman_tree(freq_table)

        # Read the rest of the file as the encoded bit stream
        encoded_bytes = f.read()
        encoded = bitarray()
        encoded.frombytes(encoded_bytes)

# Decode the bit stream

```

```
mapped = huffman_decode(encoded, tree, num_samples)

# Inverse map to get residuals

residuals = [inverse_map_residual(u) for u in mapped]
```

```
# Reconstruct the original samples from residuals
```

```
samples = np.zeros(num_samples, dtype=np.int16)
```

```
samples[0] = residuals[0]
```

```
if num_samples > 1:
```

```
    samples[1] = residuals[1] + samples[0]
```

```
for i in range(2, num_samples):
```

```
    if order == 1:
```

```
        predicted = samples[i-1]
```

```
    elif order == 2:
```

```
        predicted = 2 * samples[i-1] - samples[i-2]
```

```
    else:
```

```
        predicted = samples[i-1]
```

```
    samples[i] = predicted + residuals[i]
```

```
return samples
```

```
# Processing the Audio File with Huffman Compression
```

```
def process_audio_file_huffman(input_filename, order_values=[1, 2]):
```

```
    samples, num_channels, frame_rate, sample_width = read_wav_file(input_filename)
```

```
    original_size = os.path.getsize(input_filename)
```

```
    results = {'Original Size': original_size}
```

```
    for order in order_values:
```

```
        encoded_filename = input_filename.replace('.wav', f'_HuffOrder{order}.bin')
```

```
        compressed_size = encode_to_file_huffman(samples, order, encoded_filename)
```

```
        results[f'Huffman (Order={order})'] = compressed_size
```

```

        decoded_samples = decode_from_file_huffman(encoded_filename)

        decoded_filename = input_filename.replace('.wav', f'_HuffOrder{order}_Dec.wav')

        write_wav_file(decoded_samples, num_channels, frame_rate, sample_width,
            decoded_filename)

        if np.array_equal(decoded_samples, samples):

            print(f"Lossless compression verified for {input_filename} with predictor
            order={order}")

        else:

            print(f"Warning: Lossless compression failed for {input_filename} with
            predictor order={order}")

    for order in order_values:

        compressed_size = results[f'Huffman (Order={order})']

        percent_compression = ((original_size - compressed_size) / original_size) * 100

        results[f'% Compression (Order={order})'] = percent_compression

    return results

```

#Implementation

```

if __name__ == "__main__":

    input_files = ['Sound1.wav', 'Sound2.wav']

    order_values = [1, 2, 3]

    table_data = {}

    for input_file in input_files:

        if os.path.exists(input_file):

            results = process_audio_file_huffman(input_file, order_values)

            table_data[input_file] = results

        else:

            print(f"File {input_file} not found.")

```

```

# Print summary table

print("\nCompression Analysis Table:")

print("| Audio File | Original Size | Huffman (Order=2) | Huffman (Order=1) | %  
Compression (Order=2) | % Compression (Order=1) |")

print("|-----|-----|-----|-----|-----  
-----|-----|")

for file, data in table_data.items():

    print(f"| {file:<11} | {data['Original Size']:>13} | {data['Huffman  
(Order=2)']:>17} | {data['Huffman (Order=1)']:>17} | {data['% Compression  
(Order=2)']:>23.2f} | {data['% Compression (Order=1)']:>23.2f} |")

```

EXERCISE 3: CODE

LIST FILES IN FOLDER

```

import glob

# extract list of video files

extensions = ['mp4', 'avi', 'mov', 'mkv', 'flv']

video_files = []

for ext in extensions:

    video_files.extend(glob.glob(f"films/*.{ext}"))

print(video_files)

```

EXTRACT METADATA FROM VIDEOS

```

import subprocess

import json

import os

import pandas as pd

from IPython.display import display

```

```

def get_video_metadata(video_path):
    # Extract metadata from a video file using ffprobe

    cmd = [
        "ffprobe",
        "-v", "error",
        "-show_entries", "format:stream",
        "-of", "json",
        video_path
    ]

    result = subprocess.run(cmd, stdout=subprocess.PIPE, stderr=subprocess.PIPE,
text=True)

    if result.returncode != 0:
        print(f"Error processing {video_path}: {result.stderr}")
        return None

    return json.loads(result.stdout)


def format_frame_rate(frame_rate):
    # Convert frame rate from fraction to FPS

    try:
        num, den = map(int, frame_rate.split('/'))
        return f"{num / den:.2f} FPS"
    except:
        return "N/A"


def format_bit_rate(bit_rate, unit="Mb"):
    # Convert bit rate from bits per second to Mb/s or kb/s

    try:

```



```

    bit_rate = int(bit_rate)

    if unit == "Mb":

        return f"{bit_rate / 1_000_000:.2f} Mb/s"

    elif unit == "kb":

        return f"{bit_rate / 1_000:.2f} kb/s"

except:

    return "N/A"

```

```

def extract_metadata_from_folder(folder_path):

    # Extract relevant metadata from all video files in the given folder

    data = []

    for file in os.listdir(folder_path):

        video_path = os.path.join(folder_path, file)

        if os.path.isfile(video_path):

            metadata = get_video_metadata(video_path)

            if metadata:

                format_info = metadata.get("format", {})

                streams = metadata.get("streams", [])

                # Extract video and audio stream details

                video_stream = next((s for s in streams if s.get("codec_type") ==
"video"), {})

                audio_stream = next((s for s in streams if s.get("codec_type") ==
"audio"), {})

                width = video_stream.get("width", None)

                height = video_stream.get("height", None)

                aspect_ratio = video_stream.get("display_aspect_ratio", "N/A")

                # Compute aspect ratio manually if missing

```

```

if aspect_ratio == "N/A" and width and height:
    aspect_ratio = f"{width}:{height}"

# Extract the exact container format, fallback to file extension if
needed

detected_container = format_info.get("format_name", "N/A").split(",")[0]
file_extension = file.split(".")[1].lower()

# If detected container is ambiguous, use file extension
if detected_container == "mov":
    container = file_extension
else:
    container = detected_container

data.append({
    "File Name": file,
    "Container": container,
    "Video Codec": video_stream.get("codec_name", "N/A"),
    "Audio Codec": audio_stream.get("codec_name", "N/A"),
    "Frame Rate": format_frame_rate(video_stream.get("avg_frame_rate",
"N/A")),
    "Aspect Ratio": aspect_ratio,
    "Resolution": f"{width}x{height}" if width and height else "N/A",
    "Bit Rate (Video)": format_bit_rate(video_stream.get("bit_rate",
"N/A"), "Mb"),
    "Bit Rate (Audio)": format_bit_rate(audio_stream.get("bit_rate",
"N/A"), "kb"),
    "Channels": "Stereo" if audio_stream.get("channels", 0) == 2 else
audio_stream.get("channels", "N/A"),
})

return pd.DataFrame(data)

```

```
# Extract metadata and display in table format

films_metadata_df = extract_metadata_from_folder("films")

# Display the table in Jupyter Notebook

display(films_metadata_df)
```

FILTER OUT UNFULFILLED FORMAT & PRINT OUT IN .TXT FILE

```
import subprocess

import json

import os

import pandas as pd

from IPython.display import display


# Festival's required format

REQUIRED_FORMAT = {

    "Container": "mp4",

    "Video Codec": "h264",

    "Audio Codec": "aac",

    "Frame Rate": "25.00 FPS",

    "Aspect Ratio": "16:9",

    "Resolution": "640x360",

    "Bit Rate (Video) Min": 2.0, # Mb/s

    "Bit Rate (Video) Max": 5.0, # Mb/s

    "Bit Rate (Audio) Max": 256.0, # kb/s

    "Channels": "Stereo"

}


def get_video_metadata(video_path):

    # Extract metadata from a video file using ffprobe
```

```
cmd = [  
    "ffprobe",  
    "-v", "error",  
    "-show_entries", "format:stream",  
    "-of", "json",  
    video_path  
]
```

```
result = subprocess.run(cmd, stdout=subprocess.PIPE, stderr=subprocess.PIPE,  
text=True)
```

```
if result.returncode != 0:  
    print(f"Error processing {video_path}: {result.stderr}")  
    return None
```

```
return json.loads(result.stdout)
```

```
def format_frame_rate(frame_rate):  
    # Convert frame rate from fraction to FPS  
    try:  
        num, den = map(int, frame_rate.split('/'))  
        return f"{num / den:.2f} FPS"  
    except:  
        return "N/A"
```

```
def format_bit_rate(bit_rate, unit="Mb"):  
    # Convert bit rate from bits per second to Mb/s or kb/s  
    try:  
        bit_rate = int(bit_rate)  
        if unit == "Mb":
```



```

# Extract the exact container format, fallback to file extension if
needed

detected_container = format_info.get("format_name", "N/A").split(",")[0]

file_extension = file.split(".")[1].lower()

# If detected container is ambiguous, use file extension

container = file_extension if detected_container == "mov" else
detected_container

# Store extracted metadata

data.append({

    "File Name": file,

    "Container": container,

    "Video Codec": video_stream.get("codec_name", "N/A"),

    "Audio Codec": audio_stream.get("codec_name", "N/A"),

    "Frame Rate": format_frame_rate(video_stream.get("avg_frame_rate",
"N/A"))),

    "Aspect Ratio": aspect_ratio,

    "Resolution": f"{width}x{height}" if width and height else "N/A",

    "Bit Rate (Video)": format_bit_rate(video_stream.get("bit_rate",
"N/A")), "Mb"),

    "Bit Rate (Audio)": format_bit_rate(audio_stream.get("bit_rate",
"N/A")), "kb"),

    "Channels": "Stereo" if audio_stream.get("channels", 0) == 2 else
audio_stream.get("channels", "N/A"),

})

return pd.DataFrame(data)

def check_compliance_and_generate_report(films_metadata_df,
report_file="format_compliance_report.txt"):

```

```

# Check each film against the required format and generate a report for non-
compliant films

non_compliant_films = []

for _, row in films_metadata_df.iterrows():

    issues = []

    # Check each format requirement

    if row["Container"] != REQUIRED_FORMAT["Container"]:

        issues.append(f"Container should be {REQUIRED_FORMAT['Container']}, found
{row['Container']}")

    if row["Video Codec"] != REQUIRED_FORMAT["Video Codec"]:

        issues.append(f"Video Codec should be {REQUIRED_FORMAT['Video Codec']},
found {row['Video Codec']}")

    if row["Audio Codec"] != REQUIRED_FORMAT["Audio Codec"]:

        issues.append(f"Audio Codec should be {REQUIRED_FORMAT['Audio Codec']},
found {row['Audio Codec']}")

    if row["Frame Rate"] != REQUIRED_FORMAT["Frame Rate"]:

        issues.append(f"Frame Rate should be {REQUIRED_FORMAT['Frame Rate']}, found
{row['Frame Rate']}")

    if row["Aspect Ratio"] != REQUIRED_FORMAT["Aspect Ratio"]:

        issues.append(f"Aspect Ratio should be {REQUIRED_FORMAT['Aspect Ratio']},
found {row['Aspect Ratio']}")

    if row["Resolution"] != REQUIRED_FORMAT["Resolution"]:

        issues.append(f"Resolution should be {REQUIRED_FORMAT['Resolution']}, found
{row['Resolution']}")

```

```

        if not (REQUIRED_FORMAT["Bit Rate (Video) Min"] <= row["Bit Rate (Video)"] <=
REQUIRED_FORMAT["Bit Rate (Video) Max"]):

            issues.append(f"Video Bit Rate should be between {REQUIRED_FORMAT['Bit Rate
(Video) Min']} - {REQUIRED_FORMAT['Bit Rate (Video) Max']} Mb/s, found {row['Bit Rate
(Video)']} Mb/s")

    if row["Bit Rate (Audio)"] > REQUIRED_FORMAT["Bit Rate (Audio) Max"]:

        issues.append(f"Audio Bit Rate should be up to {REQUIRED_FORMAT['Bit Rate
(Audio) Max']} kb/s, found {row['Bit Rate (Audio)']} kb/s")

    if row["Channels"] != REQUIRED_FORMAT["Channels"]:

        issues.append(f"Audio Channels should be {REQUIRED_FORMAT['Channels']},
found {row['Channels']}")

    # If there are any issues, add them to the report

    if issues:

        non_compliant_films.append(f"Film: {row['File Name']}\n" + "\n".join(issues)
+ "\n")

# Save report

with open(report_file, "w") as f:

    if non_compliant_films:

        f.write("Non-Compliant Films Report:\n\n" + "\n".join(non_compliant_films))

    else:

        f.write("All films meet the required format.")

print(f"Report generated: {report_file}")

# Extract metadata

films_metadata_df = extract_metadata_from_folder("films")

# Display metadata table in Jupyter Notebook

```



```
display(films_metadata_df)
```

```
# Check compliance and generate report
```

```
check_compliance_and_generate_report(films_metadata_df)
```

AUTOMATICALLY CONVERT TO CORRECT FORMATS

```
import subprocess
```

```
def convert_non_compliant_films(films_metadata_df, folder_path):
```

```
    # Convert non-compliant films to the correct format using ffmpeg
```

```
    for _, row in films_metadata_df.iterrows():
```

```
        issues = []
```

```
        # Check if the film is non-compliant
```

```
        if row["Container"] != REQUIRED_FORMAT["Container"]:
```

```
            issues.append("Container format")
```

```
        if row["Video Codec"] != REQUIRED_FORMAT["Video Codec"]:
```

```
            issues.append("Video Codec")
```

```
        if row["Audio Codec"] != REQUIRED_FORMAT["Audio Codec"]:
```

```
            issues.append("Audio Codec")
```

```
        if row["Frame Rate"] != REQUIRED_FORMAT["Frame Rate"]:
```

```
            issues.append("Frame Rate")
```

```
        if row["Aspect Ratio"] != REQUIRED_FORMAT["Aspect Ratio"]:
```

```
            issues.append("Aspect Ratio")
```

```
        if row["Resolution"] != REQUIRED_FORMAT["Resolution"]:
```

```
            issues.append("Resolution")
```

```
        if not (REQUIRED_FORMAT["Bit Rate (Video) Min"] <= row["Bit Rate (Video)"] <=
REQUIRED_FORMAT["Bit Rate (Video) Max"]):
```

```
            issues.append("Video Bit Rate")
```

```
        if row["Bit Rate (Audio)"] > REQUIRED_FORMAT["Bit Rate (Audio) Max"]:
```

```

        issues.append("Audio Bit Rate")

    if row["Channels"] != REQUIRED_FORMAT["Channels"]:

        issues.append("Audio Channels")

# If there are issues, convert the file

if issues:

    input_file = os.path.join(folder_path, row["File Name"])

    output_file = os.path.join(folder_path, row["File Name"].rsplit(".", 1)[0] +
                                "_formatOK.mp4")

    print(f"Converting: {row['File Name']} (Fixing: {'', '.join(issues)}) →
{output_file}")

# ffmpeg command to convert the video

cmd = [

    "ffmpeg",

    "-i", input_file,          # Input file

    "-c:v", "libx264",         # Video codec: H.264

    "-b:v", "3M",              # Video bitrate: 3 Mb/s (within 2-5 Mb/s)

    "-vf", "scale=640:360",    # Resize to 640x360

    "-r", "25",                # Frame rate: 25 FPS

    "-aspect", "16:9",         # Aspect ratio: 16:9

    "-c:a", "aac",             # Audio codec: AAC

    "-b:a", "256k",            # Audio bitrate: up to 256 kb/s

    "-ac", "2",                # Audio channels: Stereo

    "-preset", "fast",         # Fast encoding preset

    "-y",                      # Overwrite output if exists

    output_file

]

# Run the ffmpeg command

```

```
        subprocess.run(cmd, stdout=subprocess.PIPE, stderr=subprocess.PIPE,
text=True)
```

```
    print(f"Conversion completed: {output_file}")
```

```
# Run conversion for non-compliant films
```

```
convert_non_compliant_films(films_metadata_df, "films")
```

CHECK OUTPUT FILES AFTER CONVERSION

```
import subprocess
```

```
import json
```

```
import os
```

```
import pandas as pd
```

```
from IPython.display import display
```

```
def get_video_metadata(video_path):
```

```
    # Extract metadata from a video file using ffprobe
```

```
    cmd = [
        "ffprobe",
        "-v", "error",
        "-show_entries", "format:stream",
        "-of", "json",
        video_path
    ]
```

```
    result = subprocess.run(cmd, stdout=subprocess.PIPE, stderr=subprocess.PIPE,
text=True)
```

```
    if result.returncode != 0:
```

```
        print(f"Error processing {video_path}: {result.stderr}")
```

```
        return None
```

```
return json.loads(result.stdout)
```

```
def format_frame_rate(frame_rate):  
    # Convert frame rate from fraction to FPS  
  
    try:  
        num, den = map(int, frame_rate.split('/'))  
        return f"{num / den:.2f} FPS"  
  
    except:  
        return "N/A"
```

```
def format_bit_rate(bit_rate, unit="Mb"):  
    # Convert bit rate from bits per second to Mb/s or kb/s  
  
    try:  
        bit_rate = int(bit_rate)  
  
        if unit == "Mb":  
            return f"{bit_rate / 1_000_000:.2f} Mb/s"  
  
        elif unit == "kb":  
            return f"{bit_rate / 1_000:.2f} kb/s"  
  
    except:  
        return "N/A"
```

```
def extract_metadata_from_folder(folder_path):  
    # Extract relevant metadata from all video files in the given folder  
  
    data = []  
  
    for file in os.listdir(folder_path):  
        video_path = os.path.join(folder_path, file)  
  
        if os.path.isfile(video_path):  
            metadata = get_video_metadata(video_path)
```

```

if metadata:

    format_info = metadata.get("format", {})

    streams = metadata.get("streams", [])

    # Extract video and audio stream details

    video_stream = next((s for s in streams if s.get("codec_type") ==
"video"), {})

    audio_stream = next((s for s in streams if s.get("codec_type") ==
"audio"), {})

    width = video_stream.get("width", None)

    height = video_stream.get("height", None)

    aspect_ratio = video_stream.get("display_aspect_ratio", "N/A")

    # Compute aspect ratio manually if missing

    if aspect_ratio == "N/A" and width and height:

        aspect_ratio = f"{width}:{height}"

    # Extract the exact container format, fallback to file extension if
needed

    detected_container = format_info.get("format_name", "N/A").split(",")[0]

    file_extension = file.split(".")[1].lower()

    # If detected container is ambiguous, use file extension

    if detected_container == "mov":

        container = file_extension

    else:

        container = detected_container

    data.append({

        "File Name": file,

```

```

        "Container": container,
        "Video Codec": video_stream.get("codec_name", "N/A"),
        "Audio Codec": audio_stream.get("codec_name", "N/A"),
        "Frame Rate": format_frame_rate(video_stream.get("avg_frame_rate",
"N/A"))),

        "Aspect Ratio": aspect_ratio,
        "Resolution": f"{width}x{height}" if width and height else "N/A",
        "Bit Rate (Video)": format_bit_rate(video_stream.get("bit_rate",
"N/A"), "Mb"),
        "Bit Rate (Audio)": format_bit_rate(audio_stream.get("bit_rate",
"N/A"), "kb"),
        "Channels": "Stereo" if audio_stream.get("channels", 0) == 2 else
audio_stream.get("channels", "N/A"),
    })

    return pd.DataFrame(data)

# Extract metadata and display in table format
films_metadata_df = extract_metadata_from_folder("films")

# Display the table in Jupyter Notebook
display(films_metadata_df)

```

COMPARE BEFORE AND AFTER

```

def compare_original_and_converted(folder_path):
    """Compare metadata of original and converted videos."""

    original_files = [f for f in os.listdir(folder_path) if not
f.endswith("_formatOK.mp4")]

    data = []

```

```

for file in original_files:

    original_path = os.path.join(folder_path, file)

    converted_path = os.path.join(folder_path, file.rsplit(".", 1)[0] +
    "_formatOK.mp4")

    if os.path.exists(converted_path):

        original_metadata = get_video_metadata(original_path)

        converted_metadata = get_video_metadata(converted_path)

        if original_metadata and converted_metadata:

            orig_streams = original_metadata.get("streams", [])

            conv_streams = converted_metadata.get("streams", [])

            orig_video = next((s for s in orig_streams if s.get("codec_type") ==
"video"), {})

            conv_video = next((s for s in conv_streams if s.get("codec_type") ==
"video"), {})

            orig_audio = next((s for s in orig_streams if s.get("codec_type") ==
"audio"), {})

            conv_audio = next((s for s in conv_streams if s.get("codec_type") ==
"audio"), {})

            data.append({

                "File Name": file,

                "Orig Video Codec": orig_video.get("codec_name", "N/A"),

                "Conv Video Codec": conv_video.get("codec_name", "N/A"),

                "Orig Resolution": f"{orig_video.get('width',
'N/A')}\x{orig_video.get('height', 'N/A')}",

                "Conv Resolution": f"{conv_video.get('width',
'N/A')}\x{conv_video.get('height', 'N/A')}",

                "Orig Aspect Ratio": orig_video.get("display_aspect_ratio", "N/A"),

```

```

        "Conv Aspect Ratio": conv_video.get("display_aspect_ratio", "N/A"),

        "Orig Frame Rate":
format_frame_rate(orig_video.get("avg_frame_rate", "N/A")),

        "Conv Frame Rate":
format_frame_rate(conv_video.get("avg_frame_rate", "N/A")),

        "Orig Bit Rate (Video)": format_bit_rate(orig_video.get("bit_rate",
"N/A")), "Mb"),

        "Conv Bit Rate (Video)": format_bit_rate(conv_video.get("bit_rate",
"N/A")), "Mb"),

        "Orig Audio Codec": orig_audio.get("codec_name", "N/A"),

        "Conv Audio Codec": conv_audio.get("codec_name", "N/A"),

        "Orig Bit Rate (Audio)": format_bit_rate(orig_audio.get("bit_rate",
"N/A")), "kb"),

        "Conv Bit Rate (Audio)": format_bit_rate(conv_audio.get("bit_rate",
"N/A")), "kb"),

        "Orig Channels": orig_audio.get("channels", "N/A"),

        "Conv Channels": conv_audio.get("channels", "N/A"),

    })

df_comparison = pd.DataFrame(data)

display(df_comparison)

# Run the comparison function

compare_original_and_converted("films")

```

END